Математическое моделирование

Лабораторная работа № 3

Абрикосов Артем

Содержание

1 Цель работы **6**

2 Задание **7**

3 Выполнение лабораторной работы **8**
3.1 Теоретические сведения 8
3.2 Задача 14

4 Две системы ОДУ: численное решение и визуализация **16**
4.1 Инициализация проекта и загрузка пакетов 16
4.2 Начальные условия и временной интервал 17
4.3 Параметры первой системы 17
4.4 Параметры второй системы 17
4.5 Внешние воздействия для первой системы 18
4.6 Внешние воздействия для второй системы 18
4.7 Определение моделей 18
4.8 Утилита: один прогон (решение, график, таблица, сохранение) . . 19
4.9 Запуск 1: первая система 20
4.10 Запуск 2: вторая система 20

5 Параметрическое исследование двух систем ОДУ **22**
5.1 Активация проекта и загрузка пакетов 22
5.2 Внешние воздействия для первой системы 23
5.3 Внешние воздействия для второй системы 23
5.4 Определение моделей 23
5.5 Определение базовых параметров в Dict 24
5.6 Функция-обертка для запуска одного эксперимента 25
5.7 Запуск базового эксперимента (линейная система) 27
5.8 Визуализация базового эксперимента 28
5.9 Второй базовый эксперимент (нелинейная система) 28
5.10 Параметрическое сканирование 30
5.11 Запуск всех экспериментов и сбор результатов 31
5.12 Анализ и визуализация результатов сканирования 33
5.13 Бенчмаркинг 37
5.14 Сохранение всех результатов 39
5.15 Базовые эксперименты 41

5.16 Параметрическое сканирование 43
5.17 Время вычислений 45
5.18 Анализ итоговой метрики norm_final 46

6 Выводы **47**

Список литературы **48**

Список иллюстраций

3.1 Жесткая модель войны 11
3.2 Фазовые траектории для второго случая 13

5.1 Базовый эксперимент (linear) 41
5.2 Базовый эксперимент (nonlinear) 42
5.3 Сканирование траекторий x(t) 43
5.4 Сканирование траекторий y(t) 44
5.5 Зависимость времени вычисления 45
5.6 Зависимость norm_final 46

Список таблиц

1 Цель работы

В рамках данной лабораторной работы рассматриваются элементарные математические модели вооружённого противостояния, известные как модели Ланчестера. В подобных моделях ключевым параметром выступает численность противоборствующих сторон. В конфликте могут участвовать как регулярные вооружённые силы, так и нерегулярные формирования — партизанские отряды.

Если в некоторый момент времени численность одной из сторон становится равной нулю, считается, что эта сторона потерпела поражение, при условии, что у противника остаются боеспособные силы.

2 Задание

1. Рассмотреть три варианта моделей Ланчестера.
2. Построить графики изменения численности войск во времени.
3. Определить сторону, одержавшую победу в моделируемом конфликте.

3 Выполнение лабораторной работы

3.1 Теоретические сведения

Рассмотрим три различных сценария ведения боевых действий:

1. противостояние двух регулярных армий;
2. конфликт между регулярной армией и партизанскими формированиями;
3. противоборство между партизанскими отрядами.

3.1.1 Модель регулярных армий

В случае регулярных войск изменение численности армий определяется несколькими факторами:

1. потери, не связанные непосредственно с боевыми действиями (болезни, травмы, дезертирство);
2. потери, возникающие в результате боевого взаимодействия сторон;
3. поступление подкреплений.

С учётом указанных факторов система уравнений имеет вид

$$\begin{cases} \frac{dx}{dt} = -a(t)x(t) - b(t)y(t) + P(t) \\ \frac{dy}{dt} = -c(t)x(t) - h(t)y(t) + Q(t) \end{cases}$$

Здесь члены $-a(t)x(t)$ и $-h(t)y(t)$ отражают потери, возникающие по причинам, не связанным с боем. Слагаемые $-b(t)y(t)$ и $-c(t)x(t)$ характеризуют урон, наносимый противником.

Коэффициенты $b(t)$ и $c(t)$ описывают эффективность ведения боевых действий армиями Y и X соответственно.

Функции $P(t)$ и $Q(t)$ учитывают поступление подкреплений.

3.1.2 Модель регулярной армии и партизан

В случае участия партизанских формирований характер боевых действий изменяется. Нерегулярные силы действуют рассредоточенно и скрытно, поэтому противник вынужден применять неизбирательные методы борьбы.

Темп потерь партизан зависит как от численности регулярной армии, так и от количества самих партизан. Математическая модель принимает вид

$$\begin{cases} \frac{dx}{dt} = -a(t)x(t) - b(t)y(t) + P(t) \\ \frac{dy}{dt} = -c(t)x(t)y(t) - h(t)y(t) + Q(t) \end{cases}$$

3.1.3 Модель противостояния партизанских отрядов

Если обе стороны представлены нерегулярными формированиями, система уравнений принимает форму

$$\begin{cases} \frac{dx}{dt} = -a(t)x(t) - b(t)x(t)y(t) + P(t) \\ \frac{dy}{dt} = -h(t)y(t) - c(t)x(t)y(t) + Q(t) \end{cases}$$

3.1.4 Упрощённая модель Ланчестера

В простейшем варианте предполагается, что коэффициенты b и c постоянны, а дополнительные потери и подкрепления отсутствуют.

Тогда система принимает вид

$$\begin{cases} \frac{dx}{dt} = -by \\ \frac{dy}{dt} = -ax \end{cases}$$

Здесь считается, что каждый солдат армии X за единицу времени выводит из строя c бойцов противника, а каждый солдат армии Y — b бойцов армии X.

Данная система допускает аналитическое решение.

Из уравнений следует

$$\frac{dx}{dy} = \frac{by}{cx}$$

Откуда получаем

$$cx\,dx = by\,dy$$

После интегрирования:

$$cx^2 - by^2 = C$$

Таким образом, изменение численности армий происходит вдоль гиперболических траекторий.

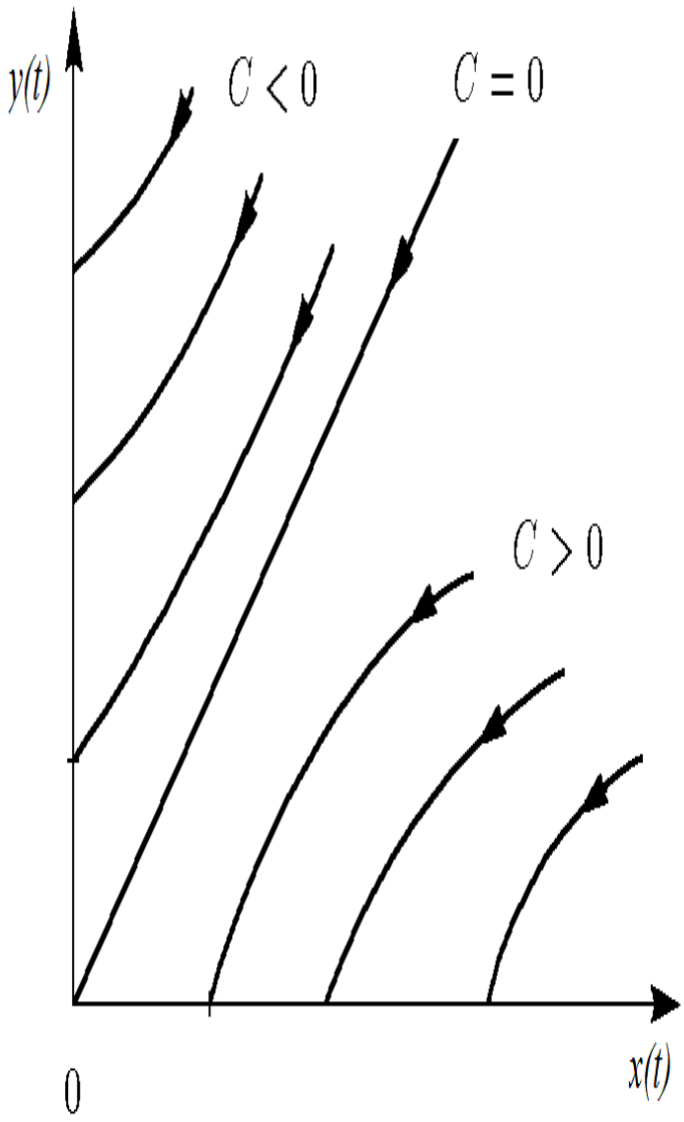

Рисунок 3.1: Жесткая модель войны

Границу между различными исходами конфликта задаёт прямая

$$\sqrt{cx} = \sqrt{by}$$

Если начальное состояние находится выше этой прямой, армия X уничтожается и победу одерживает армия Y.

Если ниже — выигрывает армия X.

При начальных условиях, лежащих на этой линии, обе армии постепенно истощаются, и конфликт затягивается на неопределённо долгое время.

Модель показывает, что для компенсации численного превосходства противника требуется значительно более эффективное вооружение. Например, для борьбы с противником, превосходящим по численности в два раза, эффективность оружия должна быть примерно в четыре раза выше.

Следует учитывать, что данная модель сильно упрощает реальную ситуацию и используется в основном для качественного анализа.

3.1.5 Упрощённая модель армии и партизан

В аналогичном упрощении система принимает вид

$$\begin{cases} \frac{dx}{dt} = -by(t) \\ \frac{dy}{dt} = -cx(t)y(t) \end{cases}$$

Эта система сводится к выражению

$$\frac{d}{dt}\left(\frac{b}{2}x^2(t) - cy(t)\right) = 0$$

Следовательно

$$\frac{b}{2}x^2(t) - cy(t) = \frac{b}{2}x^2(0) - cy(0) = C_1$$

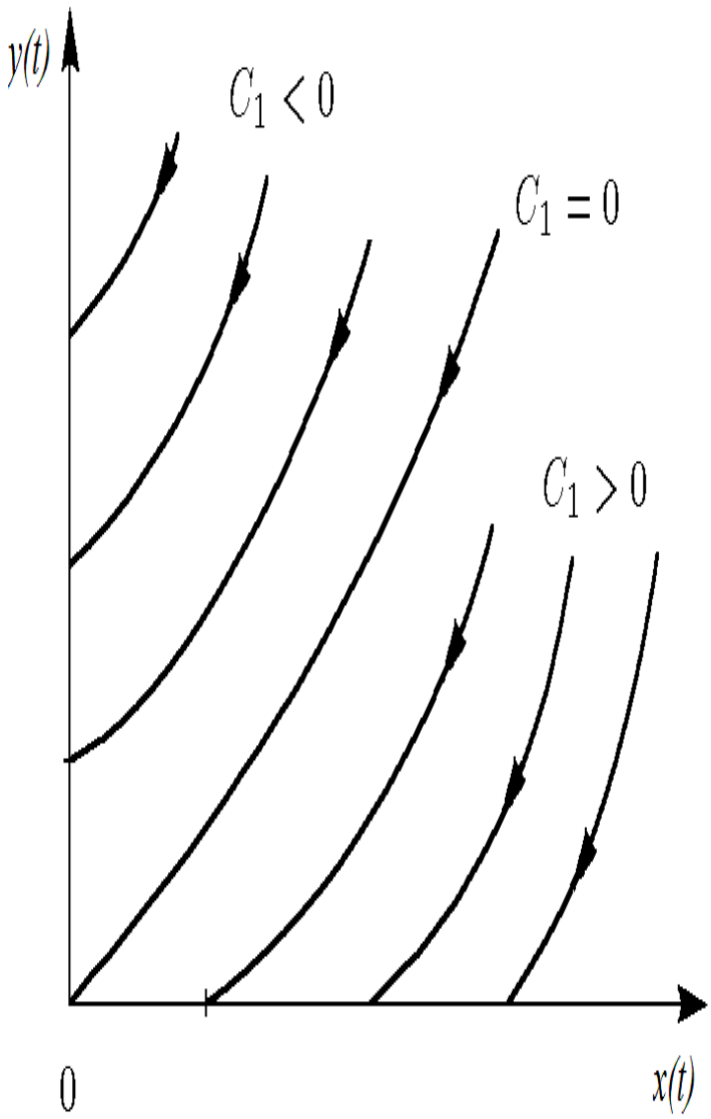

Рисунок 3.2: Фазовые траектории для второго случая

Из анализа фазовых траекторий следует:

- при $C_1 > 0$ победу одерживает регулярная армия;
- при $C_1 < 0$ преимущество получают партизанские силы.

Для победы партизан необходимо увеличить коэффициент c, характеризующий эффективность их действий, а также увеличить начальную численность.

Важно отметить, что требуемый рост численности должен происходить пропорционально квадрату численности регулярной армии $x(0)$.

Таким образом, регулярные войска в данной модели находятся в более выгодном положении.

Рассматриваемые модели представляют собой системы обыкновенных дифференциальных уравнений второго порядка, широко применяемые при описании различных процессов в естественных науках.

3.2 Задача

Между странами X и Y происходит военное противостояние. Численности армий со временем изменяются и описываются функциями $x(t)$ и $y(t)$.

В начальный момент:

- страна X располагает армией численностью 32888 человек;
- страна Y имеет армию численностью 17777 человек.

Будем считать коэффициенты a, b, c, h постоянными.

Функции $P(t)$ и $Q(t)$ являются непрерывными.

Необходимо построить графики изменения численности армий для следующих моделей.

3.2.1 1. Модель регулярных войск

$$\begin{cases} \frac{dx}{dt} = -0.55x(t) - 0.77y(t) + 1.5\sin(3t+1) \\ \frac{dy}{dt} = -0.66x(t) - 0.44y(t) + 1.2\cos(t+1) \end{cases}$$

3.2.2 2. Модель регулярной армии и партизан

$$\begin{cases} \frac{dx}{dt} = -0.27x(t) - 0.88y(t) + \sin(20t) \\ \frac{dy}{dt} = -0.68x(t)y(t) - 0.37y(t) + \cos(10t) \end{cases}$$

Для моделирования динамики и построения графиков использовались внешние файлы с программным кодом:

4 Две системы ОДУ: численное решение и визуализация

Цель: решить две системы ОДУ, построить графики решений, сохранить изображения и таблицы с результатами.

4.1 Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
using JLD2

script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE)
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

4.2 Начальные условия и временной интервал

```
x0 = 32888.0
y0 = 17777.0
t0 = 0.0
tmax = 1.0

u0 = [x0, y0]
tspan = (t0, tmax)
```

4.3 Параметры первой системы

```
a = 0.55
b = 0.77
c = 0.66
d = 0.44

p1 = (a = a, b = b, c = c, d = d)
```

4.4 Параметры второй системы

```
a2 = 0.27
b2 = 0.88
c2 = 0.68
d2 = 0.37

p2 = (a = a2, b = b2, c = c2, d = d2)
```

4.5 Внешние воздействия для первой системы

```
function P(t)
    return 1.5 * sin(3 * t + 1)
end

function Q(t)
    return 1.2 * cos(t + 1)
end
```

4.6 Внешние воздействия для второй системы

```
function P2(t)
    return sin(20 * t)
end

function Q2(t)
    return cos(10 * t) + 1
end
```

4.7 Определение моделей

Первая система: dx/dt = -ax - by + P(t) dy/dt = -cx - dy + Q(t)

```
function syst1!(dy, y, p, t)
    dy[1] = -p.a * y[1] - p.b * y[2] + P(t)
    dy[2] = -p.c * y[1] - p.d * y[2] + Q(t)
end
```

Вторая система: dx/dt = -a*x* - *b*y + P2(t) dy/dt = -c*x*y - d*y + Q2(t)

```
function syst2!(dy, y, p, t)
    dy[1] = -p.a * y[1] - p.b * y[2] + P2(t)
    dy[2] = -p.c * y[1] * y[2] - p.d * y[2] + Q2(t)
end
```

4.8 Утилита: один прогон (решение, график, таблица, сохранение)

```
function run_case(case_name; model!, u0, tspan, p, nt=100)
```

— Сетка по времени —

```
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))
```

— Решение ОДУ —

```
    prob = ODEProblem(model!, u0, tspan, p)
    sol = solve(prob, Tsit5(), saveat=tgrid)
```

— Таблица с результатами —

```
    df = DataFrame(
        t = sol.t,
        x = first.(sol.u),
        y = last.(sol.u)
    )
```

— Визуализация —

```
    plt = plot(sol,
        xlabel = "t",
        ylabel = "Значение",
        label = ["x(t)" "y(t)"],
        lw = 2,
        title = "Решение системы — $case_name",
        legend = :topright
    )
```

— Сохранение —

```
    savefig(plt, plotsdir(script_name, "$case_name.png"))
    @save datadir(script_name, "data_$case_name.jld2") df p u0 tspan nt

    return (plt = plt, df = df, sol = sol)
end
```

4.9 Запуск 1: первая система

```
res1 = run_case("system1"; model! = syst1!, u0 = u0, tspan = tspan, p = p1, nt = 100)

println("Система 1 — первые 5 строк:")
println(first(res1.df, 5))
```

4.10 Запуск 2: вторая система

```
res2 = run_case("system2"; model! = syst2!, u0 = u0, tspan = tspan, p = p2, nt = 100)

println("\nСистема 2 — первые 5 строк:")
println(first(res2.df, 5))
```

(опционально) показать последний график в интерактивной среде

```
res2.plt
```

5 Параметрическое исследование двух систем ОДУ

5.1 Активация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
```

Установка каталогов

```
script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE)
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

5.2 Внешние воздействия для первой системы

```
function P(t)
    return 1.5 * sin(3 * t + 1)
end

function Q(t)
    return 1.2 * cos(t + 1)
end
```

5.3 Внешние воздействия для второй системы

```
function P2(t)
    return sin(20 * t)
end

function Q2(t)
    return cos(10 * t) + 1
end
```

5.4 Определение моделей

Первая система: dx/dt = -ax - by + P(t) dy/dt = -cx - dy + Q(t)

```
function syst_linear!(dy, y, p, t)
    dy[1] = -p.a * y[1] - p.b * y[2] + P(t)
    dy[2] = -p.c * y[1] - p.d * y[2] + Q(t)
end
```

Вторая система: dx/dt = -a*x* - *b*y + P2(t) dy/dt = -c*x*y - d*y + Q2(t)

```
function syst_nonlinear!(dy, y, p, t)
    dy[1] = -p.a * y[1] - p.b * y[2] + P2(t)
    dy[2] = -p.c * y[1] * y[2] - p.d * y[2] + Q2(t)
end
```

5.5 Определение базовых параметров в Dict

model_type управляет выбором системы: :linear -> первая система :nonlinear -> вторая система

```
base_params = Dict(
    :x0 => 32888.0,
    :y0 => 17777.0,

    :tspan => (0.0, 1.0),
    :nt => 100,

    :model_type => :linear,         # :linear или :nonlinear
```

Параметры первой системы

```
    :a => 0.55,
    :b => 0.77,
    :c => 0.66,
    :d => 0.44,
```

Параметры второй системы

```
    :a2 => 0.27,
    :b2 => 0.88,
    :c2 => 0.68,
    :d2 => 0.37,

    :solver => Tsit5(),
    :experiment_name => "base_experiment"
)

println("Базовые параметры эксперимента:")
for (key, value) in base_params
    println(" $key = $value")
end
```

5.6 Функция-обертка для запуска одного эксперимента

```
function run_single_experiment(params::Dict)
    @unpack x0, y0, tspan, nt, model_type, a, b, c, d, a2, b2, c2, d2, solver = param

    u0 = [x0, y0]
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))

    if model_type == :linear
        p = (a=a, b=b, c=c, d=d)
        prob = ODEProblem(syst_linear!, u0, tspan, p)
    else
```

```
        p = (a=a2, b=b2, c=c2, d=d2)
        prob = ODEProblem(syst_nonlinear!, u0, tspan, p)
    end

    sol = solve(prob, solver; saveat=tgrid)

    x_vals = first.(sol.u)
    y_vals = last.(sol.u)
```

— Мини-анализ —

```
    x_final = x_vals[end]
    y_final = y_vals[end]

    x_max_abs = maximum(abs.(x_vals))
    y_max_abs = maximum(abs.(y_vals))

    norm_final = sqrt(x_final^2 + y_final^2)

    return Dict(
        "solution" => sol,
        "t_points" => sol.t,
        "x_values" => x_vals,
        "y_values" => y_vals,

        "x_final" => x_final,
        "y_final" => y_final,
        "x_max_abs" => x_max_abs,
        "y_max_abs" => y_max_abs,
```

```
        "norm_final" => norm_final,

        "parameters" => params
    )
end
```

5.7 Запуск базового эксперимента (линейная система)

```
data_base, path_base = produce_or_load(
    datadir(script_name, "single"),
    base_params,
    run_single_experiment;
    prefix = "ode",
    tag = false,
    verbose = true
)

println("\nРезультаты базового эксперимента:")
println(" x_final: ", data_base["x_final"])
println(" y_final: ", data_base["y_final"])
println(" x_max_abs: ", data_base["x_max_abs"])
println(" y_max_abs: ", data_base["y_max_abs"])
println(" norm_final: ", round(data_base["norm_final"]; digits=4))
println(" Файл результатов: ", path_base)
```

5.8 Визуализация базового эксперимента

```
p1 = plot(
    data_base["t_points"], data_base["x_values"],
    label="x(t)",
    xlabel="t",
    ylabel="Значение",
    title="Базовый эксперимент (model_type=$(base_params[:model_type]))",
    lw=2,
    legend=:topright,
    grid=true
)
plot!(
    p1,
    data_base["t_points"], data_base["y_values"],
    label="y(t)",
    lw=2
)

savefig(p1, plotsdir(script_name, "single_experiment_linear.png"))
```

5.9 Второй базовый эксперимент (нелинейная система)

```
base_params2 = copy(base_params)
base_params2[:model_type] = :nonlinear
base_params2[:experiment_name] = "base_experiment_nonlinear"
```

```
data_base2, path_base2 = produce_or_load(
    datadir(script_name, "single"),
    base_params2,
    run_single_experiment;
    prefix = "ode",
    tag = false,
    verbose = true
)

println("\nРезультаты второго базового эксперимента:")
println(" x_final: ", data_base2["x_final"])
println(" y_final: ", data_base2["y_final"])
println(" x_max_abs: ", data_base2["x_max_abs"])
println(" y_max_abs: ", data_base2["y_max_abs"])
println(" norm_final: ", round(data_base2["norm_final"]; digits=4))
println(" Файл результатов: ", path_base2)

p1b = plot(
    data_base2["t_points"], data_base2["x_values"],
    label="x(t)",
    xlabel="t",
    ylabel="Значение",
    title="Базовый эксперимент (model_type=$(base_params2[:model_type]))",
    lw=2,
    legend=:topright,
    grid=true
)
plot!(
```

```
    p1b,
    data_base2["t_points"], data_base2["y_values"],
    label="y(t)",
    lw=2
)

savefig(p1b, plotsdir(script_name, "single_experiment_nonlinear.png"))
```

5.10 Параметрическое сканирование

Сканируем параметр alpha: для linear alpha -> a для nonlinear alpha -> a2

```
param_grid = Dict(
    :x0 => [32888.0],
    :y0 => [17777.0],

    :tspan => [(0.0, 1.0)],
    :nt => [100],

    :model_type => [:linear, :nonlinear],

    :a => [0.30, 0.55, 0.80, 1.00],     # для linear
    :b => [0.77],
    :c => [0.66],
    :d => [0.44],

    :a2 => [0.15, 0.27, 0.40, 0.60],    # для nonlinear
    :b2 => [0.88],
```

```
    :c2 => [0.68],
    :d2 => [0.37],

    :solver => [Tsit5()],
    :experiment_name => ["parametric_scan"]
)

all_params = dict_list(param_grid)

println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("Исследуемые model_type: ", param_grid[:model_type])
println("Исследуемые a: ", param_grid[:a])
println("Исследуемые a2: ", param_grid[:a2])
println("="^60)
```

5.11 Запуск всех экспериментов и сбор результатов

```
all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println("Прогресс: $i/$(length(all_params)) | model_type=$(params[:model_type]) |

    data, path = produce_or_load(
        datadir(script_name, "parametric_scan"),
```

```
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    result_summary = merge(
        params,
        Dict(
            :x_final => data["x_final"],
            :y_final => data["y_final"],
            :x_max_abs => data["x_max_abs"],
            :y_max_abs => data["y_max_abs"],
            :norm_final => data["norm_final"],
            :filepath => path
        )
    )
    push!(all_results, result_summary)

    df = DataFrame(
        t = data["t_points"],
        x = data["x_values"],
        y = data["y_values"],
        model_type = fill(string(params[:model_type]), length(data["t_points"])),
        a = fill(params[:a], length(data["t_points"])),
        a2 = fill(params[:a2], length(data["t_points"]))
    )
```

```
    push!(all_dfs, df)
end
```

5.12 Анализ и визуализация результатов сканирования

```
results_df = DataFrame(all_results)
println("\nСводная таблица результатов (первые строки):")
println(first(results_df, 10))
```

Сравнительный график x(t) для всех комбинаций

```
p2 = plot(size=(900, 520), dpi=150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = params[:model_type] == :linear ?
        "linear, a=$(params[:a])" :
        "nonlinear, a2=$(params[:a2])"

    plot!(
```

```
        p2,
        data["t_points"], data["x_values"],
        label=label_text,
        lw=2,
        alpha=0.8
    )
end
plot!(
    p2,
    xlabel="t",
    ylabel="x(t)",
    title="Сканирование: траектории x(t) для разных параметров",
    legend=:outerright,
    grid=true
)
savefig(p2, plotsdir(script_name, "parametric_scan_x_comparison.png"))
```

Сравнительный график y(t) для всех комбинаций

```
p3 = plot(size=(900, 520), dpi=150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )
```

```
    label_text = params[:model_type] == :linear ?
        "linear, a=$(params[:a])" :
        "nonlinear, a2=$(params[:a2])"

    plot!(
        p3,
        data["t_points"], data["y_values"],
        label=label_text,
        lw=2,
        alpha=0.8
    )
end
plot!(
    p3,
    xlabel="t",
    ylabel="y(t)",
    title="Сканирование: траектории y(t) для разных параметров",
    legend=:outerright,
    grid=true
)
savefig(p3, plotsdir(script_name, "parametric_scan_y_comparison.png"))
```

График нормы финального состояния

```
p4 = plot(size=(900, 520), dpi=150)

linear_sub = results_df[results_df.model_type .== :linear, :]
nonlinear_sub = results_df[results_df.model_type .== :nonlinear, :]
```

```
if nrow(linear_sub) > 0
    plot!(
        p4,
        linear_sub.a, linear_sub.norm_final,
        seriestype=:scatter,
        label="linear"
    )
end

if nrow(nonlinear_sub) > 0
    plot!(
        p4,
        nonlinear_sub.a2, nonlinear_sub.norm_final,
        seriestype=:scatter,
        label="nonlinear"
    )
end

plot!(
    p4,
    xlabel="Параметр модели",
    ylabel="norm_final",
    title="Зависимость norm_final от параметра модели",
    legend=:topleft,
    grid=true
)
savefig(p4, plotsdir(script_name, "norm_final_vs_parameter.png"))
```

5.13 Бенчмаркинг

```
println("\n" * "="^60)
println("БЕНЧМАРКИНГ ДЛЯ РАЗНЫХ ПАРАМЕТРОВ")
println("="^60)

benchmark_results = []

for params in all_params
    function benchmark_run()
        u0 = [params[:x0], params[:y0]]

        if params[:model_type] == :linear
            p = (a=params[:a], b=params[:b], c=params[:c], d=params[:d])
            prob = ODEProblem(syst_linear!, u0, params[:tspan], p)
        else
            p = (a=params[:a2], b=params[:b2], c=params[:c2], d=params[:d2])
            prob = ODEProblem(syst_nonlinear!, u0, params[:tspan], p)
        end

        return solve(
            prob,
            params[:solver];
            saveat=LinRange(params[:tspan][1], params[:tspan][2], params[:nt])
        )
    end

    println("\nБенчмарк для model_type=$(params[:model_type]), a=$(params[:a]), a2=$(
```

```
    b = @benchmark $benchmark_run() samples=80 evals=1
    tsec = median(b).time / 1e9
    println(" Медианное время: ", round(tsec; digits=6), " сек")

    push!(benchmark_results, (
        model_type = string(params[:model_type]),
        a = params[:a],
        a2 = params[:a2],
        time = tsec
    ))
end

bench_df = DataFrame(benchmark_results)
```

График времени вычисления

```
p5 = plot(size=(900, 520), dpi=150)

bench_linear = bench_df[bench_df.model_type .== "linear", :]
bench_nonlinear = bench_df[bench_df.model_type .== "nonlinear", :]

if nrow(bench_linear) > 0
    plot!(
        p5,
        bench_linear.a, bench_linear.time,
        seriestype=:scatter,
        label="linear"
    )
end
```

```
if nrow(bench_nonlinear) > 0
    plot!(
        p5,
        bench_nonlinear.a2, bench_nonlinear.time,
        seriestype=:scatter,
        label="nonlinear"
    )
end

plot!(
    p5,
    xlabel="Параметр модели",
    ylabel="Время вычисления, сек",
    title="Зависимость времени решения ODE от параметров модели",
    legend=:topleft,
    grid=true
)
savefig(p5, plotsdir(script_name, "computation_time_vs_parameter.png"))
```

5.14 Сохранение всех результатов

```
@save datadir(script_name, "all_results.jld2") base_params base_params2 param_grid al
@save datadir(script_name, "all_plots.jld2") p1 p1b p2 p3 p4 p5

println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)
```

```
println("\nРезультаты сохранены в:")
println(" • data/$(script_name)/single/ - базовые эксперименты")
println(" • data/$(script_name)/parametric_scan/ - параметрическое сканирование")
println(" • data/$(script_name)/all_results.jld2 - сводные данные")
println(" • plots/$(script_name)/ - все графики")
println(" • data/$(script_name)/all_plots.jld2 - объекты графиков")
println("\nДля анализа результатов используйте:")
println(" using JLD2, DataFrames")
println(" @load \"data/$(script_name)/all_results.jld2\"")
println(" println(results_df)")
```

5.15 Базовые эксперименты

5.15.1 Линейная модель (model_type = linear)

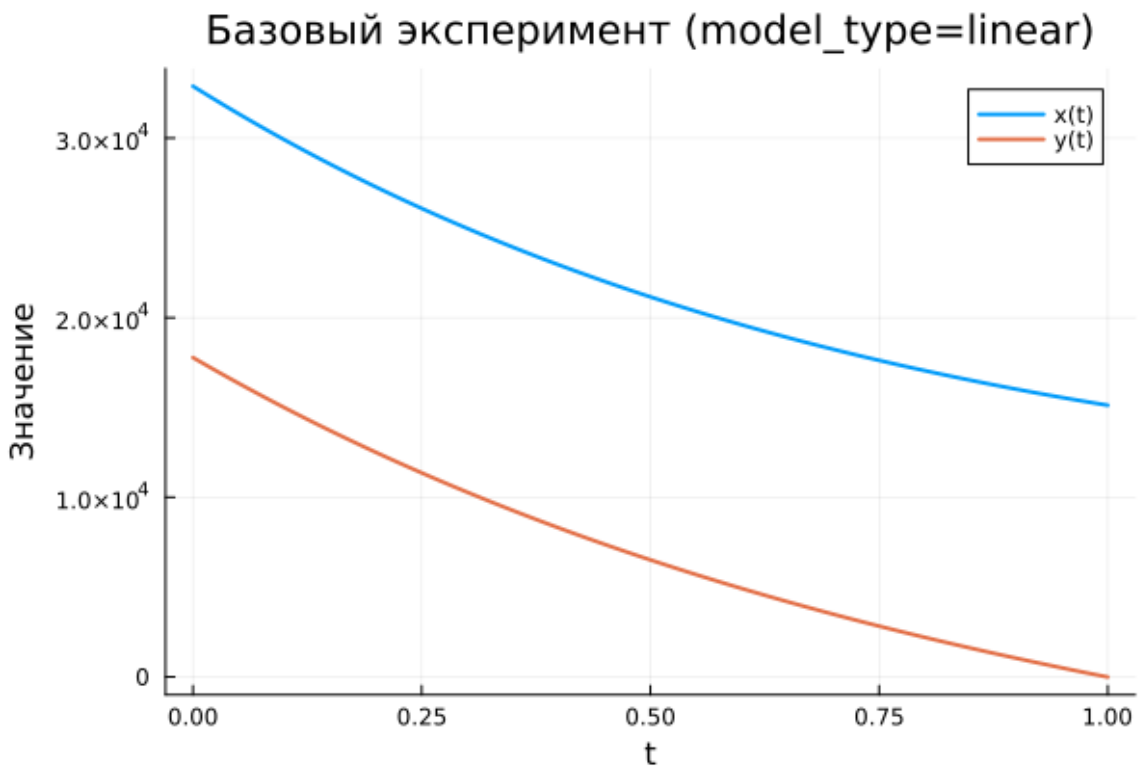

Рисунок 5.1: Базовый эксперимент (linear)

На графике представлено изменение переменных $x(t)$ и $y(t)$ во времени для линейной модели.

Обе функции демонстрируют плавное убывание по мере роста времени.

Функция $x(t)$ уменьшается постепенно и остаётся положительной на всём интервале моделирования. Это указывает на плавное ослабление системы без резких изменений.

Функция $y(t)$ убывает быстрее и к концу временного интервала практически достигает нулевого значения. Подобная динамика типична для линейных систем, где решения экспоненциально приближаются к равновесию.

5.15.2 Нелинейная модель (model_type = nonlinear)

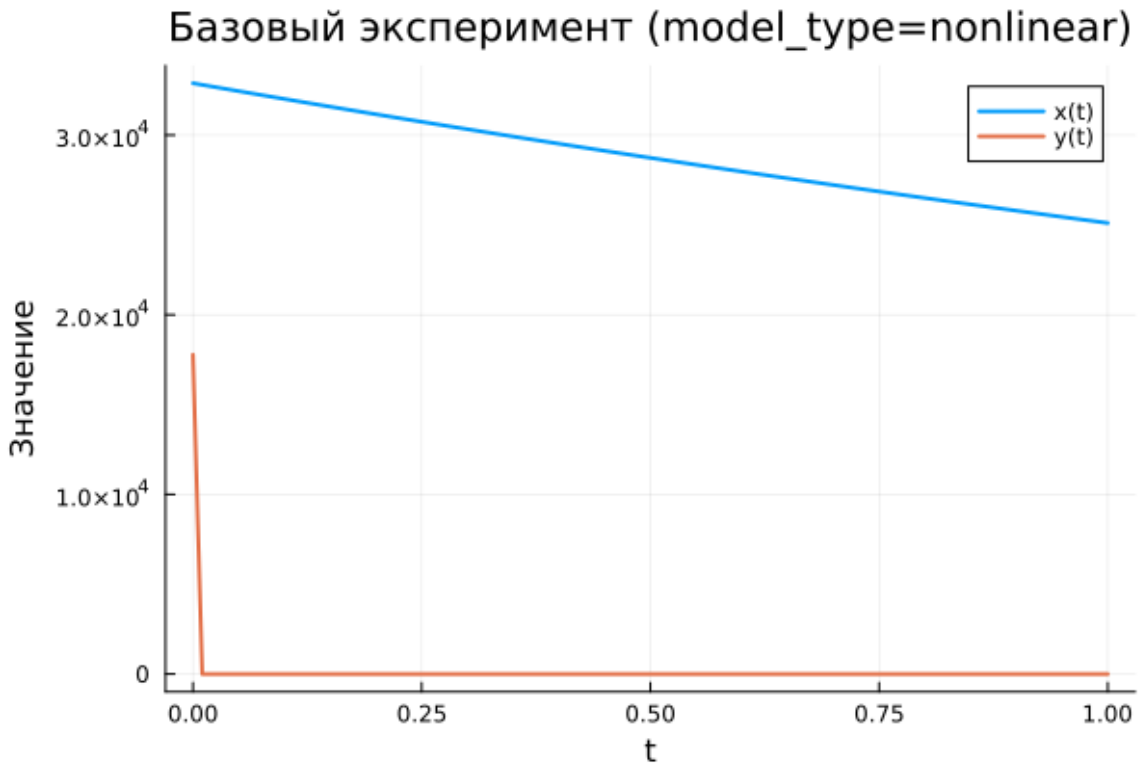

Рисунок 5.2: Базовый эксперимент (nonlinear)

В нелинейной системе наблюдается иная картина.

Переменная $x(t)$ уменьшается значительно медленнее и остаётся на сравнительно высоком уровне на протяжении всего интервала моделирования.

В то же время $y(t)$ резко уменьшается почти сразу после начала процесса и затем остаётся близкой к нулю.

Это объясняется влиянием нелинейных членов системы, которые существенно ускоряют подавление одной из переменных.

5.16 Параметрическое сканирование

5.16.1 Траектории $x(t)$ для различных параметров

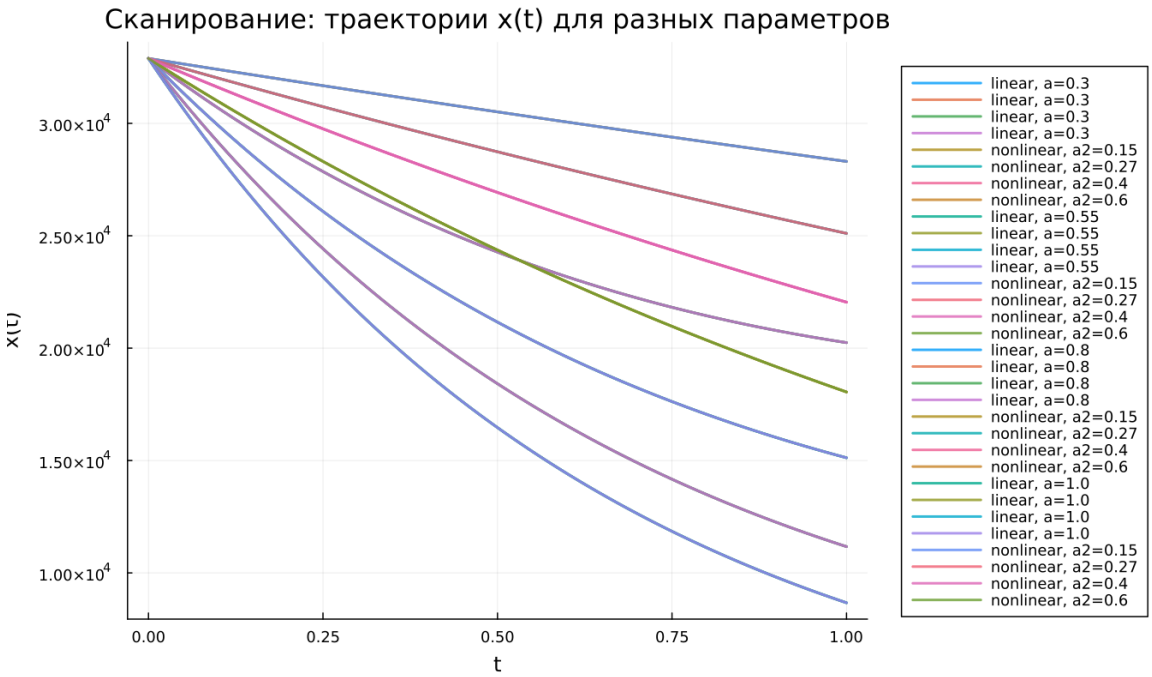

Рисунок 5.3: Сканирование траекторий x(t)

Был проведён анализ поведения системы при изменении параметров модели.

Для линейной системы варьировался параметр a, а для нелинейной — параметр a_2.

Из графиков видно, что увеличение параметра приводит к ускоренному уменьшению функции $x(t)$.

Основные наблюдения:

- при небольших значениях параметра спад происходит медленно;
- при больших значениях траектория становится значительно круче;
- различия между решениями хорошо заметны уже на средних значениях времени.

В нелинейной системе зависимость также сохраняется, однако форма траекторий становится более сложной.

5.16.2 Траектории $y(t)$ для различных параметров

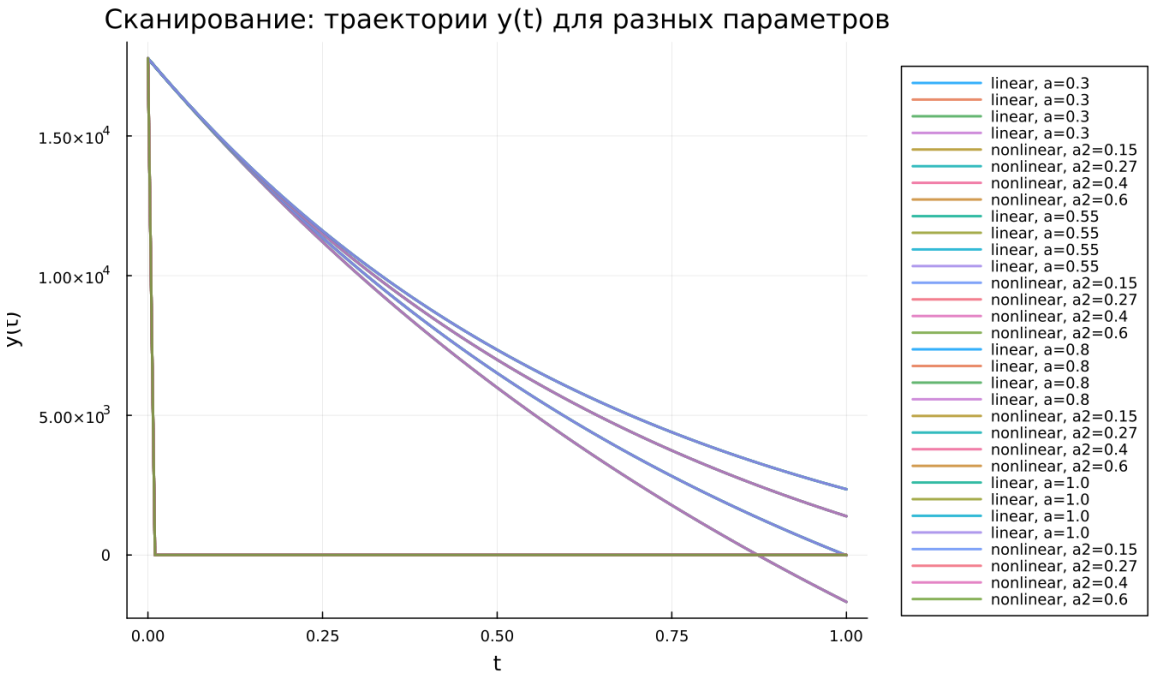

Рисунок 5.4: Сканирование траекторий y(t)

Для переменной $y(t)$ была выполнена аналогичная процедура.

В линейной модели увеличение параметра приводит к более быстрому уменьшению функции и ускоряет её приближение к нулю.

В нелинейной системе значение $y(t)$ практически сразу становится близким к нулю независимо от параметров, что указывает на сильное влияние нелинейных взаимодействий.

5.17 Время вычислений

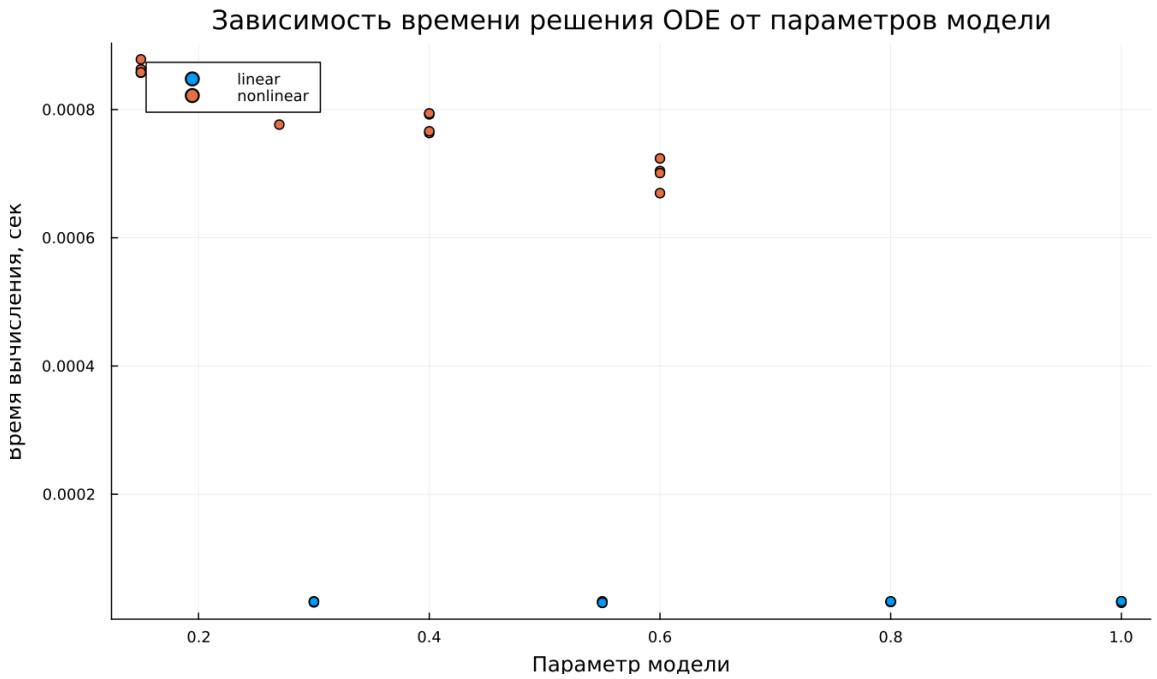

Рисунок 5.5: Зависимость времени вычисления

Было проведено сравнение времени вычисления решений системы дифференциальных уравнений.

Полученные результаты показывают:

- линейная модель решается значительно быстрее;
- время вычисления линейной системы составляет порядка 10^{-5} секунд;
- нелинейная модель требует больше времени — примерно $7 \cdot 10^{-4}$–$9 \cdot 10^{-4}$ секунд.

Несмотря на это, общее время вычислений остаётся очень малым и не создаёт ограничений для практического моделирования.

5.18 Анализ итоговой метрики norm_final

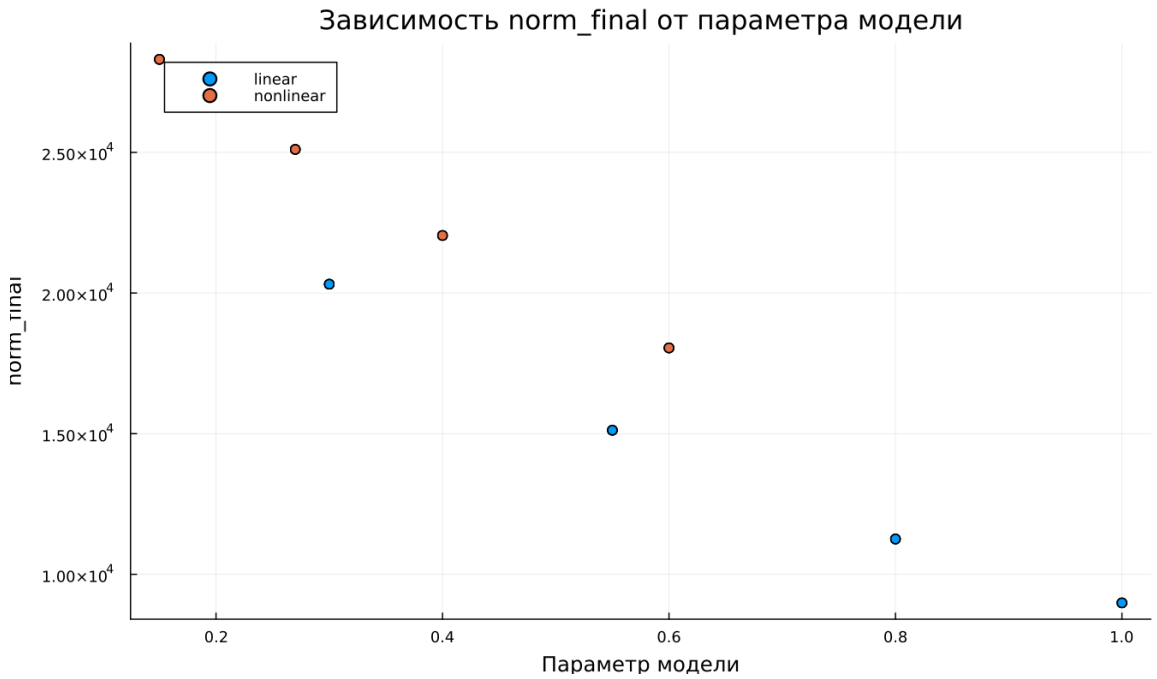

Рисунок 5.6: Зависимость norm_final

Для анализа состояния системы в конце интервала моделирования использовалась метрика

$$\text{norm_final} = \sqrt{x(t_{final})^2 + y(t_{final})^2}$$

Она характеризует величину состояния системы в финальный момент времени.

Анализ зависимости показывает:

- при увеличении параметра значение метрики уменьшается;
- линейная модель быстрее приближается к состоянию покоя;
- нелинейная система сохраняет более высокие значения метрики.

Это означает, что в линейной системе затухание происходит быстрее.

6 Выводы

1. Линейная модель характеризуется плавным и устойчивым уменьшением обеих переменных. Функции $x(t)$ и $y(t)$ монотонно уменьшаются со временем.

2. В нелинейной системе динамика существенно отличается: переменная $y(t)$ быстро уменьшается почти до нуля, после чего поведение системы определяется в основном переменной $x(t)$.

3. Изменение параметров оказывает заметное влияние на скорость уменьшения решений, особенно в линейной модели.

4. В нелинейной системе влияние параметров на $y(t)$ выражено слабее из-за сильного подавления этой переменной нелинейными членами.

5. Численные расчёты показывают, что линейная модель вычисляется быстрее, однако даже для нелинейной системы время решения остаётся крайне малым.

6. Итоговая метрика norm_final уменьшается при увеличении параметров модели, что свидетельствует об усилении затухания динамики системы.

Полученные результаты соответствуют ожидаемому поведению линейных и нелинейных дифференциальных моделей и подтверждают корректность проведённого численного моделирования.

Список литературы

1. Законы Осипова — Ланчестера
2. Дифференциальные уравнения динамики боя
3. Элементарные модели боя